

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Single-Node Hadoop and Spark Deployment with Distributed Word Count

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Big Data Processing

Contents

1 Purpose and submission context 2

2 Sixteen conceptual questions 2

3 Part A: HDFS installation and verification 2

4 Part B: Spark Standalone and HDFS integration 3

5 Final report requirements 3

1 Purpose and submission context

The first Big Data Analytics assignment, version 1.0, belongs to the first semester of academic year 1404-1405. The instructor is Dr Iman Gholampour; the assignment designers are Hossein Soleimani and Ali Habibi. The exercise combines concise conceptual answers with a documented Linux deployment of HDFS and Spark. A single machine may serve as NameNode, DataNode, Spark master, and Spark worker. This is a laboratory deployment, not evidence of a production multi-machine cluster.

2 Sixteen conceptual questions

1. Explain batch and streaming processing in Spark and give appropriate scenarios for each.
2. Define an RDD and identify workloads where it remains preferable to DataFrame or Dataset APIs.
3. Compare DataFrames and Datasets, including advantages, disadvantages, and language constraints.
4. Describe the responsibilities of the driver and executors.
5. Explain shuffle, why it becomes a performance bottleneck, and give an example.
6. Explain when cache/persist helps and compare persistence storage levels.
7. Identify causes of executor OutOfMemory and practical remedies.
8. Describe Adaptive Query Execution and the optimizations it provides.
9. Explain Tungsten and whole-stage code generation.
10. Compare Standalone, YARN, and Kubernetes deployment and their appropriate contexts.
11. Explain why Parquet and ORC can outperform CSV, discussing column statistics and compression.
12. Describe Delta Lake capabilities such as ACID transactions, time travel, and Z-ordering.
13. Discuss when a traditional data warehouse or a lakehouse is appropriate.
14. Compare FIFO and Fair scheduling with examples.
15. Explain scheduler pools and how to separate ETL from ad hoc queries.
16. Explain how dynamic allocation can assist resource sharing.

3 Part A: HDFS installation and verification

Use hadoop_installation.md and cite the exact steps in the report. Download and extract Hadoop using Step 1; configure environment variables using Step 2. Configure core-site.xml and hdfs-site.xml using Steps 3 and 4. Create dfs.namenode.name.dir and dfs.datanode.data.dir and set ownership. Set JAVA_HOME in hadoop-env.sh following Step 5. Initialize a new laboratory NameNode with hdfs namenode -format, start the services with start-dfs.sh, and report cluster health using hdfs dfsadmin -report. Formatting is a one-time initialization of a new namespace; it must not be repeated against an existing dataset. Deliver the DataNode/capacity report screenshot and the following upload/listing test, substituting the actual user name:

```
printf 'hello spark\nhello hdfs\nspark spark\n' > sample.txt
hdfs dfs -mkdir -p /user/<your_user>/lab/
hdfs dfs -put -f sample.txt /user/<your_user>/lab/
hdfs dfs -ls /user/<your_user>/lab/
```

4 Part B: Spark Standalone and HDFS integration

Use spark_installation.md and identify the Install Java and Install Spark steps. Install Java, download/extract Spark, and add the environment variables to ~/.bashrc. Start the master with start-master.sh and the worker with start-worker.sh spark://<MASTER_IP>:7077. Inspect http://<MASTER_IP>:8080 and supply a screenshot showing the registered worker. Open a Scala shell with the standalone master or local[*]. Load the HDFS test file and execute:

```
val rdd = sc.textFile("hdfs://<NAMENODE_IP>:9000/user/<your_user>/lab/
sample.txt")
val counts = rdd.flatMap(_.split("\\s+"))
.filter(_.nonEmpty).map(w => (w.toLowerCase, 1))
.reduceByKey(_ + _).sortBy { case (_, c) => -c }
counts.collect().foreach(println)
```

For the stated sample text the correct counts are spark=3, hello=2, and hdfs=1. The assignment's example prints spark=2, which omits one occurrence on the third line. Sorting equal counts needs an explicit secondary key if deterministic tie ordering is required.

5 Final report requirements

Reference the guide file and section for every installation stage. Include requested command outputs and screenshots; briefly explain encountered errors and remedies; answer every conceptual question. Preserve the installation steps and troubleshooting narrative rather than presenting only a final success screenshot.